

```
[bash]$ ls -l /dev/null
crw-rw-rw- 1 root root 1, 3 Jul 11 20:47 /dev/null
```

In the above output there are two numbers separated by a comma (1 and 3). Here, '1' is the major and '3' is the minor number. The major number identifies the driver associated with the device, i.e., which driver is to be used. The minor number is used by the kernel to determine exactly which device is being referred to. For instance, a hard disk may have three partitions. Each partition will have a separate minor number but only one major number, because the same storage driver is used for all the partitions.

Older kernels used to have a separate major number for each driver. But modern Linux kernels allow multiple drivers to share the same major number. For instance, `/dev/full`, `/dev/null`, `/dev/random` and `/dev/zero` use the same major number but different minor numbers. The output below illustrates this:

```
[bash]$ ls -l /dev/full /dev/null /dev/random /dev/zero
crw-rw-rw- 1 root root 1, 7 Jul 11 20:47 /dev/full
crw-rw-rw- 1 root root 1, 3 Jul 11 20:47 /dev/null
crw-rw-rw- 1 root root 1, 8 Jul 11 20:47 /dev/random
crw-rw-rw- 1 root root 1, 5 Jul 11 20:47 /dev/zero
```

The kernel uses the `dev_t` type to store major and minor numbers. `dev_t` type is defined in the `<linux/types.h>` header file. Given below is the representation of `dev_t` type from the header file:

```
#ifndef _LINUX_TYPES_H
#define _LINUX_TYPES_H

#define __EXPORTED_HEADERS__
#include <uapi/linux/types.h>

typedef __u32 __kernel_dev_t;

typedef __kernel_dev_t dev_t;
```

`dev_t` is an unsigned 32-bit integer, where 12 bits are used to store the major number and the remaining 20 bits are used to store the minor number. But don't try to extract the major and minor numbers directly. Instead, the kernel provides MAJOR and MINOR macros that can be used to extract the major and minor numbers. The definition of the MAJOR and MINOR macros from the `<linux/kdev_t.h>` header file is given below:

```
#ifndef _LINUX_KDEV_T_H
#define _LINUX_KDEV_T_H

#include <uapi/linux/kdev_t.h>

#define MINORBITS 20
```

```
#define MINORMASK ((1U << MINORBITS) - 1)

#define MAJOR(dev) ((unsigned int) ((dev) >> MINORBITS))
#define MINOR(dev) ((unsigned int) ((dev) & MINORMASK))
```

If you have major and minor numbers and you want to convert them to the `dev_t` type, the `MKDEV` macro will do the needful. The definition of the `MKDEV` macro from the `<linux/kdev_t.h>` header file is given below:

```
#define MKDEV(ma,mi) (((ma) << MINORBITS) | (mi))
```

We now know what major and minor numbers are and the role they play. Let us see how we can allocate major numbers. Here is the prototype of the `register_chrdev()`:

```
int register_chrdev(unsigned int major, const char *name,
struct file_operations *fops);
```

This function registers a major number for character devices. Arguments of this function are self-explanatory. The *major* argument implies the major number of interest, *name* is the name of the driver and appears in the `/proc/devices` area and, finally, *fops* is the pointer to the `file_operations` structure.

Certain major numbers are reserved for special drivers; hence, one should exclude those and use dynamically allocated major numbers. To allocate a major number dynamically, provide the value zero to the first argument, i.e., `major == 0`. This function will dynamically allocate and return a major number.

To deallocate an allocated major number use the `unregister_chrdev()` function. The prototype is given below and the parameters of the function are self-explanatory:

```
void unregister_chrdev(unsigned int major, const char *name)
```

The values of the major and name parameters must be the same as those passed to the `register_chrdev()` function; otherwise, the call will fail.

File operations

So we know how to allocate/deallocate the major number, but we haven't yet connected any of our driver's operations to the major number. To set up a connection, we are going to use the `file_operations` structure. This structure is defined in the `<linux/fs.h>` header file.

Each field in the structure must point to the function in the driver that implements a specific operation, or be left NULL for unsupported operations. The example given below illustrates that.

Without discussing lengthy theory, let us write our first 'null' driver, which mimics the functionality of a `/dev/null` pseudo device. Given below is the complete working code for the 'null' driver.

Open a file using your favourite text editor and save the code given below as `null_driver.c`: